

Pairing Based Cryptography

Bilinear pairing

- $(p, \mathbb{G}, g, \mathbb{G}_T, e)$
 - $\mathbb{G}$ and $\mathbb{G}_T$ are both multiplicative cyclic groups of order p , g is the generator of $\mathbb{G}$.

$\mathbb{G}$:base group, $\mathbb{G}_T$ target group, $e: \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$

1. Bilinear: $e(P \cdot P', Q) = e(P, Q) \cdot e(P', Q)$
 $e(P, Q \cdot Q') = e(P, Q) \cdot e(P, Q')$
2. Non-degeneracy: $e(g, g) \neq 1$ is a generator of $\mathbb{G}_T$
3. e is efficiently computable

- Pairing: $e(P^x, Q^y) = e(P, Q)^{xy}$
- Example: $e(g^x, g^y) = e(g, g)^{xy} = e(g^{xy}, g)$

Decisional Diffie-Hellman (DDH) Assumption

Assumption: we say that DDH is hard in group $\mathbb{G}$ if for all PPT algorithm A , there is a negligible function that

$$\Pr[A(\mathbb{G}, q, g^x, g^y, g^z) = 1] - \Pr[A(\mathbb{G}, q, g^x, g^y, g^{xy}) = 1] \leq \text{negl}(n)$$

where g is a generator, q is the order of the group, $x, y, z \in \mathbb{Z}_q$ are chosen uniformly.

Computational Diffie-Hellman (CDH)

Assumption: we say that CDH is hard in group $\mathbb{G}$ if for all PPT algorithm A , there is a negligible function that

$$\Pr[A(\mathbb{G}, q, g^x, g^y) = g^{xy}] \leq \text{negl}(n)$$

where g is a generator, q is the order of the group, $x, y \in \mathbb{Z}_q$ are chosen uniformly.

Bilinear pairing

Given g^x and g^y , a pairing can check that some element $h = g^{xy}$ without knowing x and y

$$e(g^x, g^y) = e(g, g)^{xy} = e(g^{xy}, g) \stackrel{?}{=} e(h, g)$$

DDH is easy in the base group, but CDH is still hard

Key Exchanges for 3 Parties

Diffie-Hellman Key exchange:

- 1) Alice selects secret a , sends g^a to Bob.
- 2) Bob selects secret b , sends g^b to Alice.
- 3) Each can compute the key g^{ab}

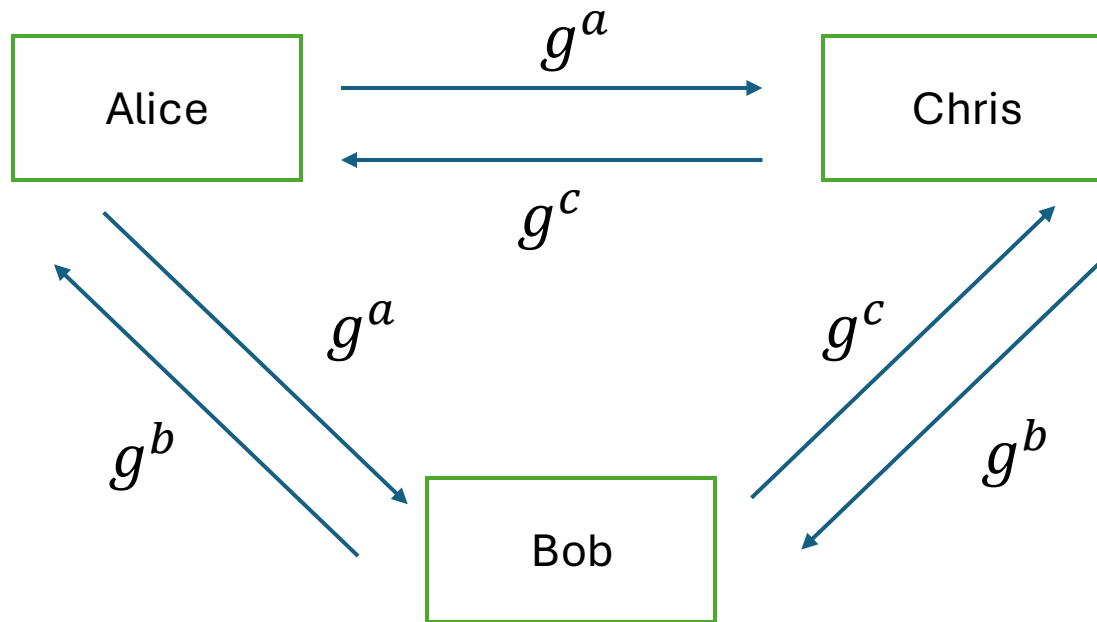
Key Exchanges for 3 Parties

Extend this with to 3 Parties **One-Round** Key Exchange ?

- 1) Alice selects secret a , sends g^a to Bob.
- 2) Bob selects secret b , sends g^b to Alice.
- 3) Each can compute the key g^{ab}

Key Exchanges for 3 Parties

Extend this with to 3 Parties **One-Round** Key Exchange



All can compute g^{abc} using bilinear pairing

Bilinear Diffie-Hellman (BDH) problem:

BDH is hard in group $\mathbb{G}$ if for all PPT algorithm A , there is a negligible function that

$$\Pr[A(\mathbb{G}, q, g^x, g^y, g^z, g^{xyz}) = 1] - \Pr[A(\mathbb{G}, q, g^x, g^y, g^z, g^r) = 1] \leq \text{negl}(n)$$

Discrete-log based signature

- Keygen: $p, \mathbb{G}, g$
private key x , public key g^x
- Sign(sk, m): $H(m)^x$, where H is a cryptographic hash that maps the message space to $\mathbb{G}$
- Verify(σ, m): $\sigma = H(m)^x$???

BLS signature [Boneh–Lynn–Shacham'2001]

- Keygen: $p, \mathbb{G}, g, \mathbb{G}_T, e$
private key x , public key g^x
- Sign(sk, m): $H(m)^x$, where H is a cryptographic hash that maps the message space to $\mathbb{G}$
- Verify(σ, m): $e(H(m), g^x) = e(\sigma, g)$
- **Signature aggregation**: aggregate n signers' signature into one signature

Correctness and security

- $e(H(m), g^x) = e(\sigma, g) = e(H(m)^x, g)$
- Security: CDH, Random Oracle

3 types of pairing

$$e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$$

- Type 1: $\mathbb{G}_1 = \mathbb{G}_2$
- Type 2: $\mathbb{G}_1 \neq \mathbb{G}_2$, but efficiently computable isomorphism $\mathbb{G}_2 \rightarrow \mathbb{G}_1$
- Type 3: $\mathbb{G}_1 \neq \mathbb{G}_2$, no efficiently computable isomorphism ✓

Additive notation

- $(p, \mathbb{G}_1, \mathbb{G}_2, g_1, g_2, \mathbb{G}_T, e)$
 - $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are both cyclic group of order p , g_1 is the generator of $\mathbb{G}_1$ and g_2 is the generator of $\mathbb{G}_2$
 $\mathbb{G}_1, \mathbb{G}_2$: base groups, $\mathbb{G}_T$ target group, $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$
1. Bilinear: $e(P \cdot P', Q) = e(P, Q) \cdot e(P', Q)$
 $e(P, Q \cdot Q') = e(P, Q) \cdot e(P, Q')$

Additive notation

- $(p, \mathbb{G}_1, \mathbb{G}_2, g_1, g_2, \mathbb{G}_T, e)$
 - $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are both cyclic group of order p , g_1 is the generator of $\mathbb{G}_1$ and g_2 is the generator of $\mathbb{G}_2$
 $\mathbb{G}_1, \mathbb{G}_2$:base groups, $\mathbb{G}_T$ target group, $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$
1. Bilinear: $e(P + P', Q) = e(P, Q) \cdot e(P', Q)$
 $e(P, Q + Q') = e(P, Q) \cdot e(P, Q')$

Additive notation

- $(p, \mathbb{G}_1, \mathbb{G}_2, g_1, g_2, \mathbb{G}_T, e)$
 - $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are both cyclic group of order p , g_1 is the generator of $\mathbb{G}_1$ and g_2 is the generator of $\mathbb{G}_2$
 $\mathbb{G}_1, \mathbb{G}_2$: base groups, $\mathbb{G}_T$ target group, $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$

1. Bilinear:
$$\begin{aligned} e(P \cdot P', Q) &= e(P, Q) \cdot e(P', Q) \\ e(P, Q \cdot Q') &= e(P, Q) \cdot e(P, Q') \end{aligned}$$

- Pairing:
$$e(P^x, Q^y) = e(P, Q)^{xy}$$

- Example:
$$e(g_1^x, g_2^y) = e(g_1, g_2)^{xy} = e(g_1^{xy}, g_2)$$

Additive notation

- $(p, \mathbb{G}_1, \mathbb{G}_2, g_1, g_2, \mathbb{G}_T, e)$
 - $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are both cyclic group of order p , g_1 is the generator of $\mathbb{G}_1$ and g_2 is the generator of $\mathbb{G}_2$
 $\mathbb{G}_1, \mathbb{G}_2$: base groups, $\mathbb{G}_T$ target group, $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$

1. Bilinear:
$$e(P + P', Q) = e(P, Q) \cdot e(P', Q)$$
$$e(P, Q + Q') = e(P, Q) \cdot e(P, Q')$$

- Pairing:
$$e([x]P, [y]Q) = e(P, Q)^{xy}$$

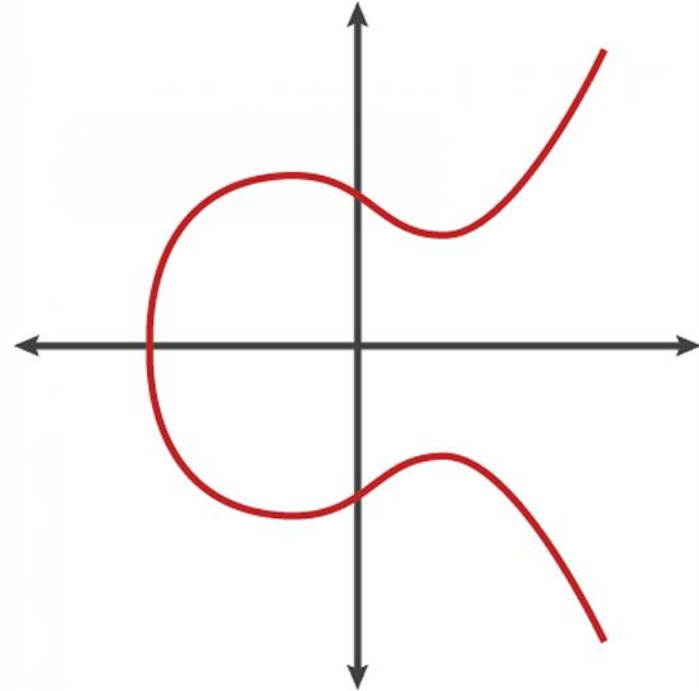
- Example:
$$e([x]g_1, [y]g_2) = e(g_1, g_2)^{xy} = e([xy]g_1, g_2)$$

Elliptic curves

$$y^2 = x^3 + ax + b \mod p$$

Example: $y^2 = x^3 + 3x + 3 \mod 7$

$$(1,0), (3,2), (3,5), (4,3), (4,4)$$



Group from Elliptic curve

$$\mathbb{G} = \{(x, y) \mid x, y \in \mathbb{Z}_p, y^2 = x^3 + ax + b \bmod p\} \cup \{O\}$$

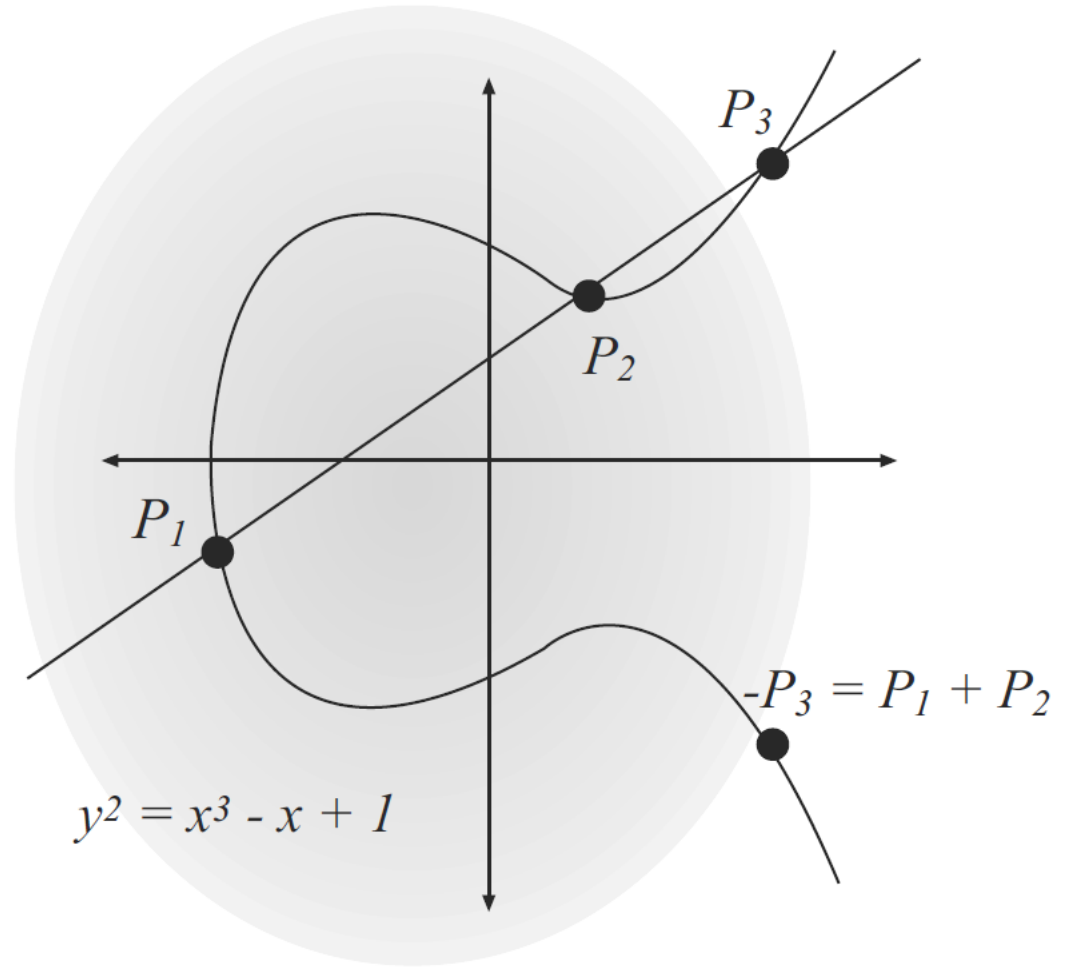
- O : point at infinity, additive identity

Point addition

- $P_1: (x_1, y_1), P_2: (x_2, y_2)$

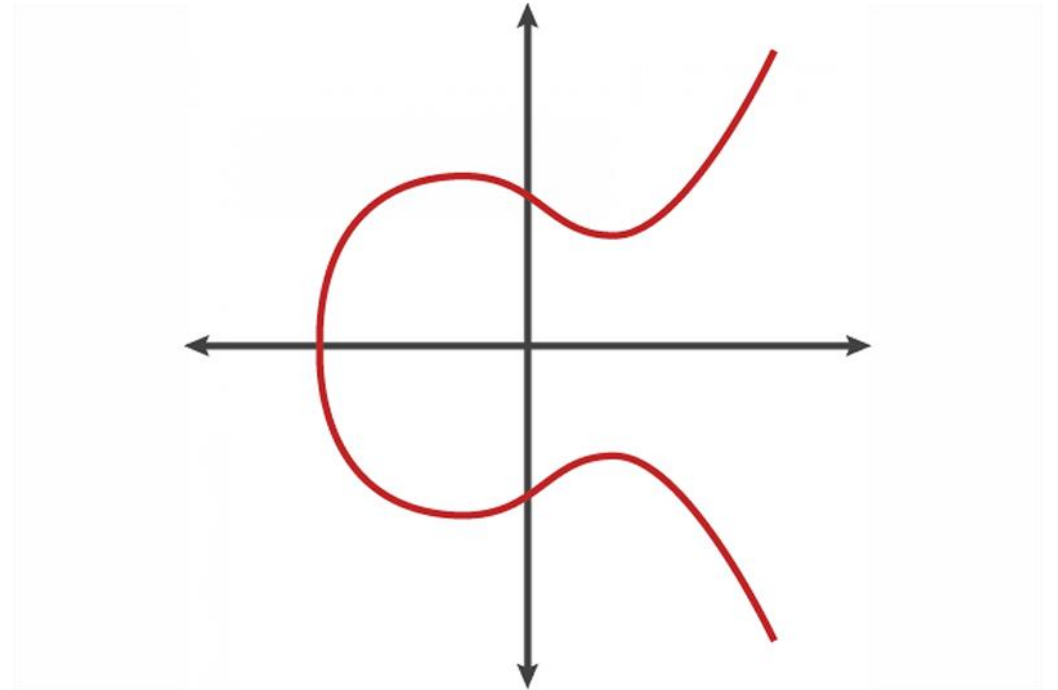
$$s = \frac{y_2 - y_1}{x_2 - x_1} \bmod p$$

$$P_3: \begin{aligned} x_3 &= s^2 - x_1 - x_2 \bmod p \\ y_3 &= s(x_3 - x_1) - y_1 \bmod p \end{aligned}$$



Inverse and identity

- $P: (x, y) \quad -P: (x, -y)$
- $P + (-P) = O$
- O : point at infinity, additive identity



Point doubling

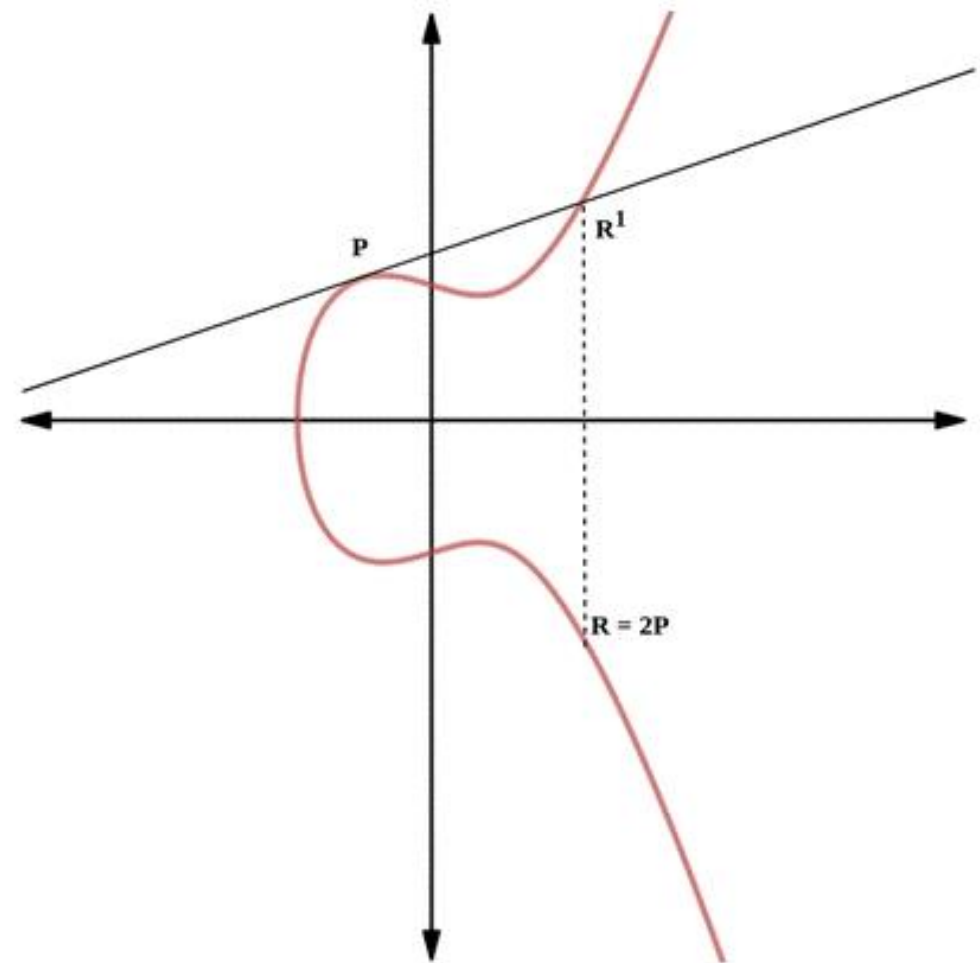
- $P: (x_1, y_1)$

$$s = \frac{3x_1^2 + a}{2y_1} \mod p$$

$$2P: x = s^2 - 2x_1 \mod p$$

$$y = s(x - x_1) - y_1 \mod p$$

$$y^2 = x^3 + ax + b \mod p$$

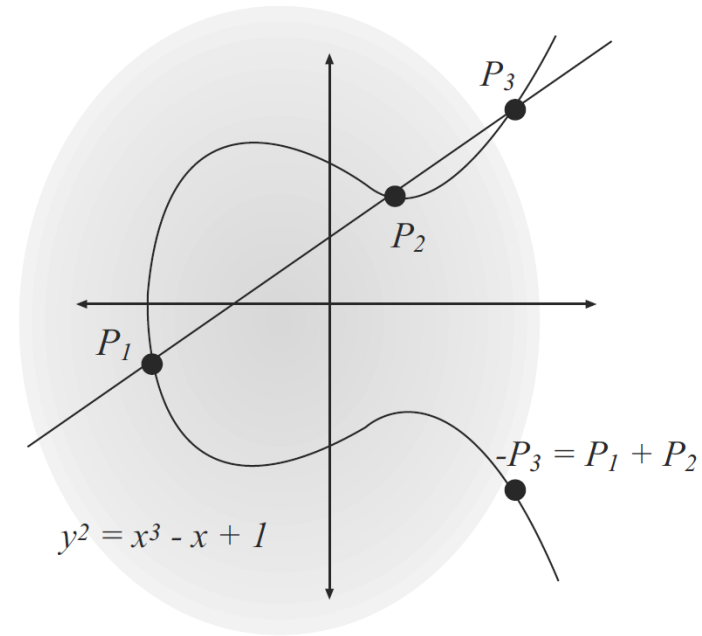


Group from Elliptic curve

$$\mathbb{G} = \{(x, y) \mid x, y \in \mathbb{Z}_p, y^2 = x^3 + ax + b \bmod p\} \cup \{O\}$$

- O : point at infinity, additive identity

1. Point addition
2. Point doubling
3. Identity and inverse: $P: (x, y) \quad -P: (x, -y) \quad P + (-P) =$
4. $P_1 = P_2$ and $y = 0, 2P_1 = O$



$\mathbb{G}$ is an abelian group under point addition

Group

Group: under 1 operation $\circ$

1. **Closure**: For all a, b in G , the result of the operation, $a \circ b$, is also in G
2. **Associativity**: For all a, b and c in G , $(a \circ b) \circ c = a \circ (b \circ c)$
3. **Identity element**: There exists an element e in G such that, for every element a in G , the equation $e \circ a = a \circ e = a$ holds. Such an element is unique
4. **Inverse element**: For each a in G , there exists an element b in G , such that $a \circ b = b \circ a = e$, where e is the identity element

Commutativity: If for all a, b in G , $a \circ b = b \circ a$, the group is called **abelian**

Secure curves

- **With proper choice of parameters**, the best known discrete log algorithm in an elliptic curve group of size q runs in time $O(\sqrt{q})$.
- 256-bit q gives 128-bit security

Number of points (order) in the group

- **Hasse bound:** let p be prime, and E be an elliptic curve over $\mathbb{Z}_p$, the order of the group $p + 1 - 2\sqrt{p} \leq \mathbf{q} \leq p + 1 + 2\sqrt{p}$

Recommended key length

Effective key length (bits)	RSA (N) (bits)	Discrete-log $\mathbb{Z}_p^*$ (bits)	Discrete-log Elliptic curve group order q
112	2048	2048	224
128	3072	3072	256
192	7680	7680	384
256	15360	15360	512

Figure 10.1, chapter 10.4

Insecure curves

- If q is composite, and all its prime factors are less than some bound q_{max} , then there is an algorithm to compute discrete log in time $O(\sqrt{q_{max}})$
- If $q = p$, discrete log solvable in polynomial time
- If q divides $p^\tau - 1$, reduce to $\mathbb{F}_{p^\tau}$

Curves in practice

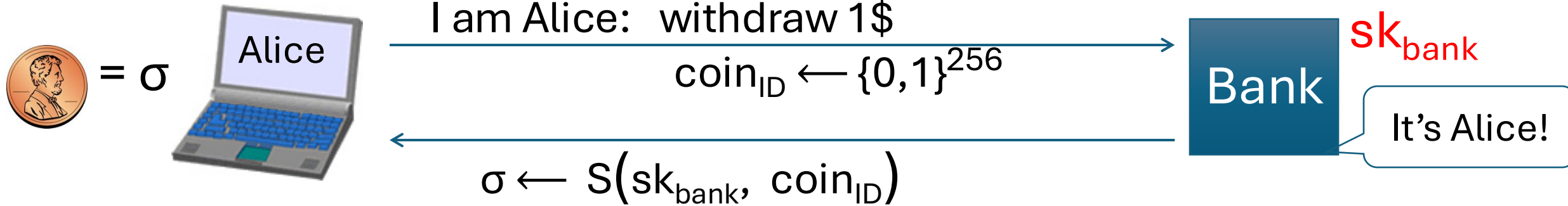
- P-256 curve:
 - $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$
- Curve 25519
 - $p = 2^{255} - 19$
- Secp256k1 curve
 - $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$

Curves in practice

- P-256 curve:
 - $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$
 - $y^2 = x^3 - 3x + b \mod p$
 - Order q is prime that is of the same magnitude as p
- Curve 25519
 - $p = 2^{255} - 19$
 - Order q is not a prime, but work with subgroup of large prime order
- Secp256k1 curve (used by Bitcoin)
 - $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$
 - $y^2 = x^3 + 7 \mod p$

Blind Signature

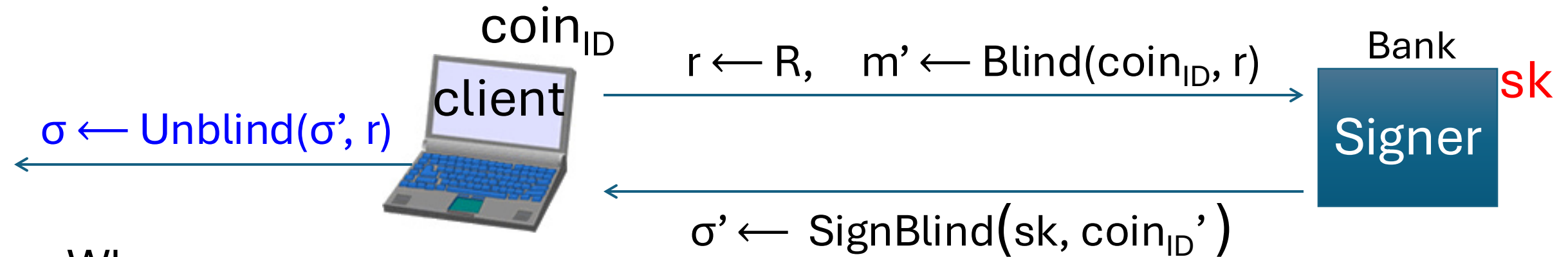
Digital Cash



- Cash is the paper with bank' signature
- Retain the physical attributes of cash
 - Unforgeable
 - Untraceable

Solution: blind signatures

Goal: we want Bank to sign coin_{ID} , but without knowing coin_{ID}



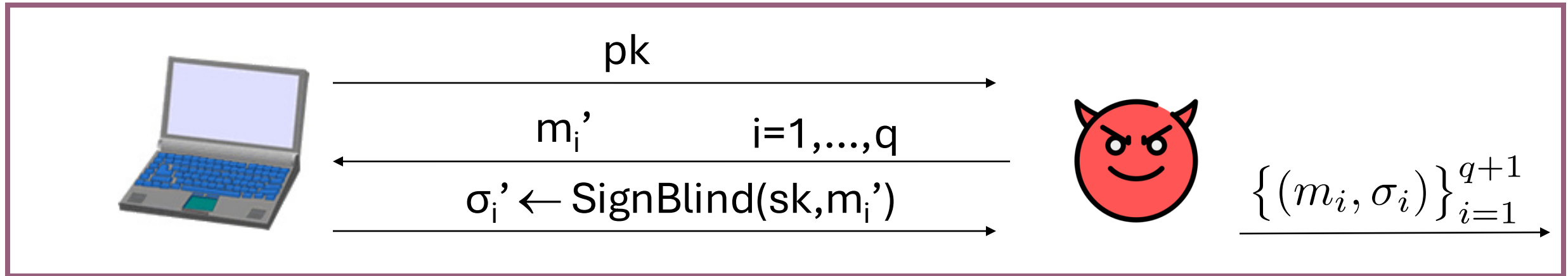
Where:

- (1) σ is a valid signature on m : $V(pk, m, \sigma) = \text{"accept"}$
- (2) $m' \leftarrow \text{Blind}(m)$ is independent of m
 - That is, m' reveals no "information" about m

Blind signatures: security

New definition of existential forgery:

adversary asks for q blind signatures, and
outputs $(q+1)$ message/signature pairs

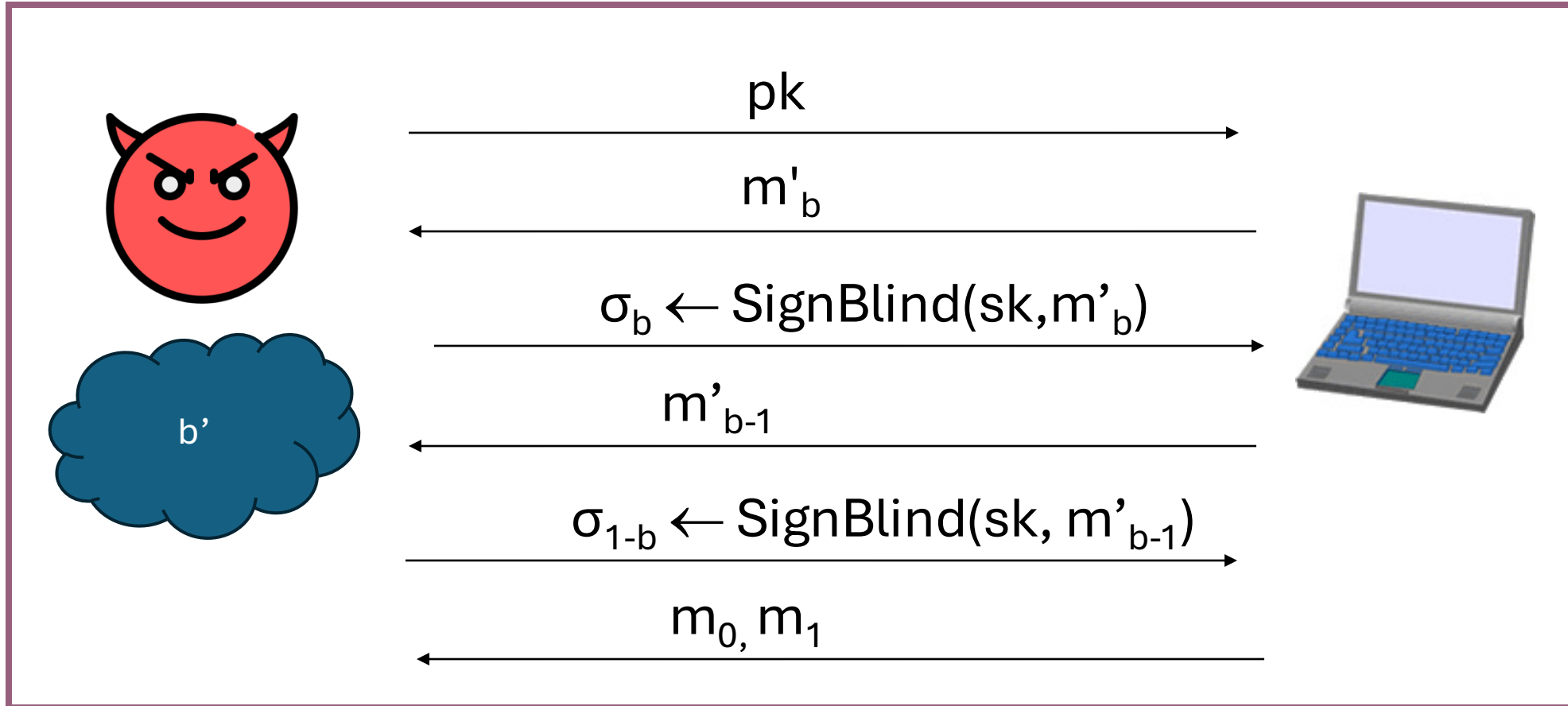


A wins if $V(pk, m_i, \sigma_i) = \text{'accept'}$ for all $i=1, \dots, q+1$

Security: for all “efficient” A , $\text{Adv}_{\text{Blind}}[A, SS] = \Pr[A \text{ wins}] \leq \text{negl}$

Blind signatures: security

Definition of blindness:



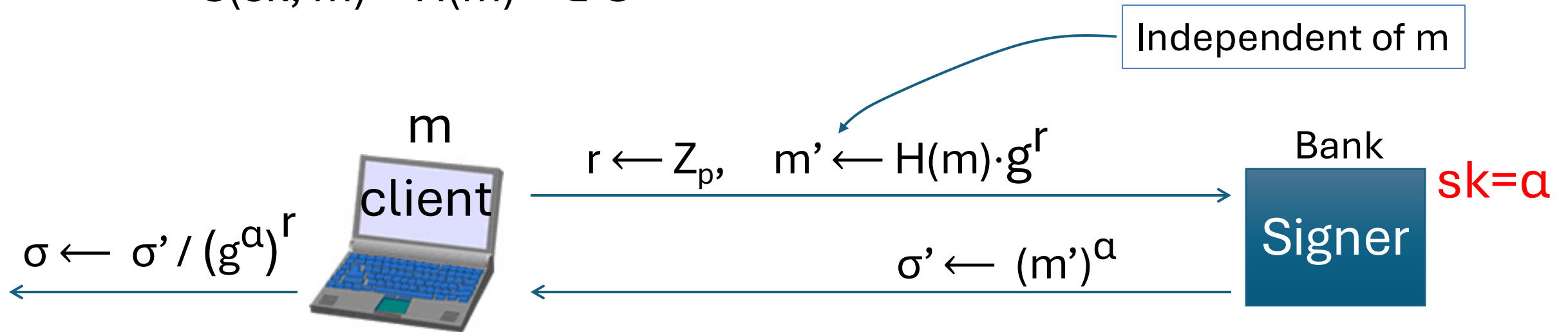
Blindness: $\text{Adv}_{\text{Blind}}[A, b' = b] = \Pr[A \text{ wins}] \leq \text{negl}$

Constructions

BLS review: G finite group of order p with a pairing

$$sk = \alpha \in \mathbb{Z}_p, \quad pk = (g, g^\alpha), \quad H: M \rightarrow G$$

$$S(sk, m) = H(m)^\alpha \in G$$



$$\text{Indeed: } \sigma = (m')^\alpha / (g^\alpha)^r = \sigma$$

Same method also works for RSA. Problem: security under strong assumption.